

DOMAIN DRIVEN DESIGN USING JAVA

Documentação Final do Backend

Sistema de Gestão - Turma do Bem

NOME DA SOLUÇÃO

Sistema de Gestão - Turma do Bem

NOME DA EQUIPE

Equipe de Desenvolvimento Turma do Bem

INTEGRANTES

Carlos Aurelio Tolosa Bianchi — RM
567897 — Turma 1tdsps
Vinicius Morrone Lustosa — RM
566884 — Turma 1tdsps

STACK PRINCIPAL

Java 17, Quarkus 3, PostgreSQL,
React, Vite, TypeScript

GitHub do projeto: <https://github.com/Carlos-Bianchi/java-challenge-04>

1. Objetivo e escopo do projeto

Este projeto representa a evolução de uma aplicação web estática para uma solução integrada, com frontend em React + Vite + TypeScript e backend em Java + Quarkus. O sistema foi desenhado para apoiar a operação da ONG Turma do Bem, conectando pacientes em situação de vulnerabilidade com dentistas voluntários por meio de um fluxo estruturado de cadastro, triagem, match, comunicação, agendamento e registro clínico.

No escopo da disciplina de Domain Driven Design Using Java, esta entrega consolida a camada de back-end responsável pela persistência, regras de negócio, validações, tratamento de exceções e exposição de endpoints REST necessários para o funcionamento da aplicação. A API foi construída em arquitetura modular com entidades, DTOs, repositórios, serviços e recursos REST, mantendo aderência ao banco de dados PostgreSQL já modelado para o projeto.

Cadastro de usuários

Match paciente-dentista

Comunicação multicanal

Dashboard analítico

Agendamentos

Registros de atendimento

2. Principais funcionalidades implementadas

- **Gestão de usuários:** cadastro, consulta, atualização, ativação/desativação e remoção de pacientes, dentistas voluntários e administradores.
- **Autenticação:** login com verificação de credenciais e resposta padronizada para integração com o frontend.
- **Sistema de match:** recomendação de dentistas com base em especialidade e disponibilidade de deslocamento, além de criação e atualização de matches.
- **Comunicação:** criação, leitura e remoção de mensagens operacionais ou vinculadas a matches.
- **Agendamentos:** criação e manutenção de consultas vinculadas exclusivamente a matches confirmados.
- **Relatórios clínicos:** registro de atendimento com atualização do status do agendamento e acompanhamento do tratamento.
- **Dashboard:** consolidação de métricas operacionais para acompanhamento da base de usuários, matches, comunicações e agenda.
- **Validações e exceções:** DTOs validados, envelope único de erro e regras de negócio com retorno consistente para o frontend.

3. Métodos lógicos principais

A aplicação possui mais de quatro rotinas com lógica de negócio. Abaixo estão os quatro métodos centrais exigidos, com explicação do papel de cada um no fluxo do sistema e um recorte do código-fonte real utilizado no backend.

3.1 `UsuarioService.registerUser(...)`

Este método centraliza o cadastro de novos usuários. Ele impede duplicidade de e-mail e CPF, gera o hash da senha, define o status inicial do usuário e cria automaticamente os registros derivados de endereço e papel específico. Para pacientes, exige especialidade e descrição da necessidade clínica. Para dentistas voluntários, exige CRO, especialidade, clínica e turno preferencial.

```
public Usuario registerUser(UsuarioRequest request) {
    if (usuarios.existsByEmail(request.email())) {
        throw new BusinessException(Response.Status.CONFLICT, "duplicate_email", "Email já cadastrado");
    }
    if (usuarios.existsByCpf(request.cpf())) {
        throw new BusinessException(Response.Status.CONFLICT, "duplicate_cpf", "CPF já cadastrado");
    }
    Usuario usuario = new Usuario();
    applyUserFields(usuario, request);
    usuario.senhaHash = passwords.hash(request.senha());
    usuario.status = request.status() == null ? StatusUsuario.pendente : request.status();
    usuarios.save(usuario);
    createAddressIfPresent(usuario, request);
    createRoleRecord(usuario, request);
    return usuario;
}
```

3.2 `MatchService.recommendDentistsForPatient(...)`

Essa rotina monta a lista de dentistas recomendados para um paciente específico. O cálculo leva em conta a especialidade necessária e a flexibilidade de deslocamento do paciente, produzindo um percentual de compatibilidade que ordena os resultados. O método usa apenas dentistas disponíveis e prontos para receber novos pacientes.

```
public List<DentistRecommendationResponse> recommendDentistsForPatient(Long patientId) {
    Paciente paciente = pacientes.findByUsuarioId(patientId);
    return dentistas.listAvailable().stream()
        .map(dentista -> recommendation(paciente, dentista))

    .sorted(Comparator.comparing(DentistRecommendationResponse::percentualCompatibilidade).reversed())
    .toList();
}

private DentistRecommendationResponse recommendation(Paciente paciente, DentistaVoluntario dentista) {
    boolean sameSpecialty = paciente.especialidadeNecessaria != null &&
    dentista.especialidadePrincipal != null
        && paciente.especialidadeNecessaria.id.equals(dentista.especialidadePrincipal.id);
    short specialty = (short) (sameSpecialty ? 70 : 25);
    short location = (short) (Boolean.TRUE.equals(paciente.aceitaDeslocamento) ? 30 : 15);
}
```

```

        BigDecimal percent = BigDecimal.valueOf(specialty +
location).min(BigDecimal.valueOf(100)).setScale(2, RoundingMode.HALF_UP);
        return new DentistRecommendationResponse(..., percent, location, specialty);
    }

```

3.3 MatchService.updateMatchStatus(...)

Este método protege a consistência do fluxo operacional. Matches finalizados não podem ser alterados novamente, e a transição para o estado *pendente* não é aceita como atualização válida. Ao registrar uma resposta, o backend também armazena a data de retorno para auditoria e rastreabilidade do processo.

```

public Match updateMatchStatus(Long matchId, StatusMatch status) {
    Match match = matches.findById(matchId);
    if (match.status == StatusMatch.cancelado || match.status == StatusMatch.confirmado) {
        throw new BusinessException(Response.Status.CONFLICT, "invalid_match_transition",
"Matches finalizados não podem mudar de status");
    }
    if (status == StatusMatch.pendente) {
        throw new BusinessException(Response.Status.BAD_REQUEST, "invalid_match_transition",
"Status pendente não é uma transição válida");
    }
    match.status = status;
    match.respondidoEm = LocalDateTime.now();
    return match;
}

```

3.4 RegistroAtendimentoService.recordAttendance(...)

O registro clínico só pode ocorrer quando existe um agendamento válido vinculado a um match confirmado. O método também impede o uso de agendamentos cancelados e, ao concluir o atendimento, atualiza automaticamente o status do agendamento para *concluída*.

```

public RegistroAtendimento recordAttendance(RegistroAtendimentoRequest request) {
    RegistroAtendimento registro = new RegistroAtendimento();
    apply(registro, request);
    registro.agendamento.status = StatusAgendamento.concluida;
    return registros.save(registro);
}

private void apply(RegistroAtendimento registro, RegistroAtendimentoRequest request) {
    Agendamento agendamento = agendamentos.findById(request.agendamentoId());
    if (agendamento.match.status != StatusMatch.confirmado) {
        throw new BusinessException(Response.Status.CONFLICT, "match_not_confirmed", "Atendimento
exige match confirmado");
    }
    if (agendamento.status == StatusAgendamento.cancelada) {
        throw new BusinessException(Response.Status.CONFLICT, "appointment_cancelled",
"Agendamento cancelado não pode receber atendimento");
    }
    registro.agendamento = agendamento;
    ...
}

```

Método adicional relevante: **DashboardService.buildDashboardSummary()** agrega métricas da operação com consultas ao banco e contadores por status, alimentando a visão analítica do módulo administrativo.

4. Tabela de endpoints da API RESTful

Recurso	URI	Verbo	Descrição	Status esperados
Autenticação	/api/auth/login	POST	Autentica usuário por e-mail e senha.	200, 400, 401
Autenticação	/api/auth/register	POST	Registra novo usuário na plataforma.	201, 400, 409
Dashboard	/api/dashboard/summary	GET	Retorna métricas agregadas da operação.	200, 500
Especialidades	/api/especialidades	GET	Lista especialidades odontológicas.	200, 500
Usuários	/api/usuarios	GET	Lista usuários cadastrados.	200, 500
Usuários	/api/usuarios/{id}	GET	Busca usuário específico por ID.	200, 404
Usuários	/api/usuarios	POST	Cria usuário com regras por papel.	201, 400, 409
Usuários	/api/usuarios/{id}	PUT	Atualiza dados cadastrais do usuário.	200, 400, 404
Usuários	/api/usuarios/{id}/status	PATCH	Atualiza o status do usuário.	200, 400, 404
Usuários	/api/usuarios/{id}	DELETE	Remove usuário existente.	204, 404
Match	/api/matches	GET	Lista matches cadastrados.	200, 500
Match	/api/matches/{id}	GET	Busca match por ID.	200, 404
Match	/api/matches/recommendations?patientId={id}	GET	Recomenda dentistas para um paciente.	200, 400, 404
Match	/api/matches	POST	Cria novo match.	201, 400, 404
Match	/api/matches/{id}/status	PATCH	Atualiza status do match.	200, 400, 404, 409
Match	/api/matches/{id}	DELETE	Remove match.	204, 404
Comunicação	/api/comunicacoes	GET	Lista mensagens e notificações.	200, 500
Comunicação	/api/comunicacoes/{id}	GET	Busca comunicação por ID.	200, 404
Comunicação	/api/comunicacoes	POST	Cria nova comunicação.	201, 400, 404

Recurso	URI	Verbo	Descrição	Status esperados
Comunicação	/api/comunicacoes/{id}/read	PATCH	Marca comunicação como lida.	200, 404
Comunicação	/api/comunicacoes/{id}	DELETE	Remove comunicação.	204, 404
Agendamentos	/api/agendamentos	GET	Lista agendamentos.	200, 500
Agendamentos	/api/agendamentos/{id}	GET	Busca agendamento por ID.	200, 404
Agendamentos	/api/agendamentos	POST	Cria agendamento para match confirmado.	201, 400, 404, 409
Agendamentos	/api/agendamentos/{id}	PUT	Atualiza agendamento.	200, 400, 404, 409
Agendamentos	/api/agendamentos/{id}	DELETE	Remove agendamento.	204, 404
Registros	/api/registros-atendimento	GET	Lista registros clínicos.	200, 500
Registros	/api/registros-atendimento/{id}	GET	Busca registro por ID.	200, 404
Registros	/api/registros-atendimento	POST	Cria registro de atendimento.	201, 400, 404, 409
Registros	/api/registros-atendimento/{id}	PUT	Atualiza registro clínico.	200, 400, 404, 409
Registros	/api/registros-atendimento/{id}	DELETE	Remove registro clínico.	204, 404

5. Protótipo com prints das telas implementadas

As imagens abaixo foram capturadas da aplicação SPA em execução local. Elas demonstram a interface usada pelo usuário final e o alinhamento entre frontend e backend do ecossistema Turma do Bem.



Sistema de Gestão - Turma do Bem

Plataforma integrada para otimizar e centralizar a operação da ONG, conectando voluntários e pacientes de forma eficiente. Transformando sorrisos e vidas através da tecnologia e solidariedade.

→ [Conhecer o Sistema](#)

Nossos Módulos Principais



Gestão de Cadastros

Sistema completo para cadastro e gerenciamento de pacientes, dentistas voluntários e funcionários da ONG com controle de acesso personalizado.



Sistema de Match

Algoritmo inteligente que conecta pacientes aos voluntários mais adequados baseado em localização, disponibilidade e necessidade.



Comunicação

Centralização de toda comunicação, garantindo visibilidade organizada e direcionamento eficiente das mensagens para as áreas correspondentes.

Funcionalidades do Sistema



Agendamento Online

Sistema inteligente de agendamento que otimiza a disponibilidade dos voluntários com as necessidades dos pacientes.



Dashboard Analítico

Visualização em tempo real de métricas e indicadores de desempenho, permitindo tomada de decisões baseadas em dados concretos.



Segurança de Dados

Proteção completa com repositórios locais, backup automatizado e criptografia para garantir a privacidade das informações.



Prontuário Eletrônico

Registro digital completo do histórico de atendimento, tratamentos realizados e acompanhamento de cada paciente.



Acesso Mobile

Interface responsiva que permite acesso completo ao sistema em qualquer dispositivo, a qualquer hora e lugar.



Criação de Relatórios

Módulo responsável pela criação de relatórios personalizados com base nas informações do sistema que o usuário desejar.

Impacto da ONG

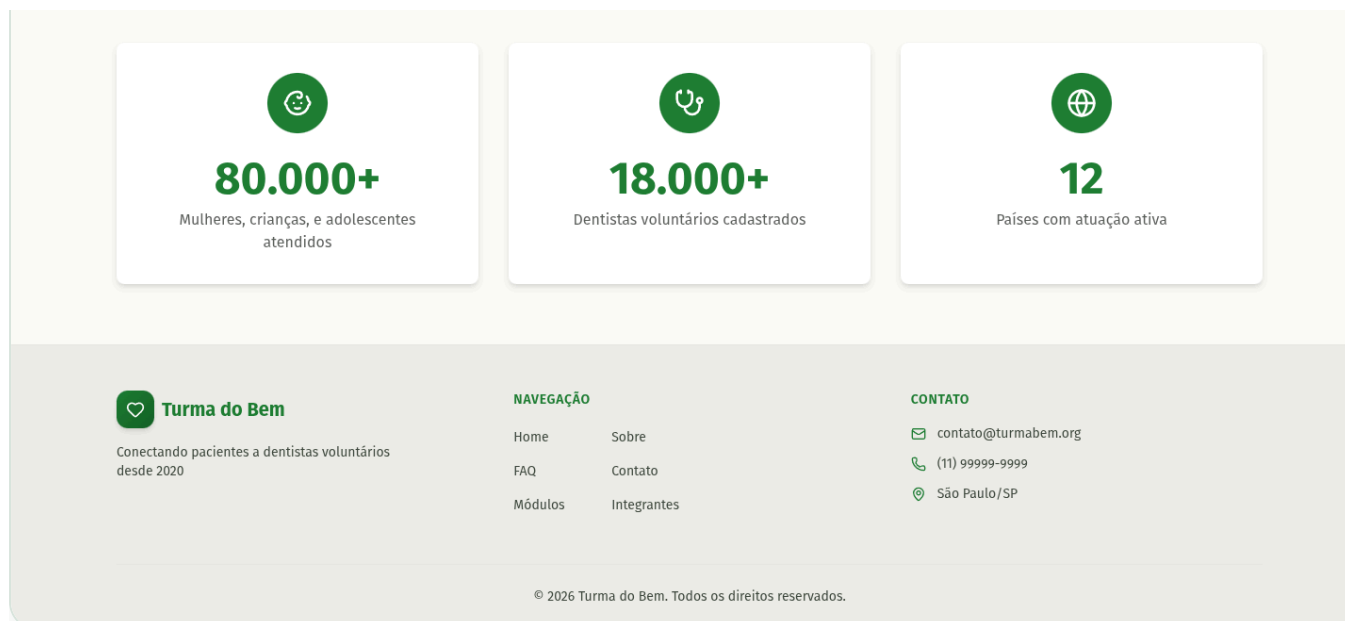


Figura 1 — Página inicial com identidade visual da solução, navegação pública e apresentação institucional.



Carregando dashboard...



Figura 2 — Dashboard analítico para acompanhamento de métricas operacionais e visão geral da plataforma.



Fila de Espera

12 Pendentes

Prioridade e necessidades

Maria Santos

São Paulo

Ortodontia

Ver Detalhes

João Oliveira

Rio de Janeiro

Implantes

Ver Detalhes

Ana Costa

Belo Horizonte

Estética

Ver Detalhes

Ver Fila Completa



TOP 3 Recomendados

Baseado no perfil do paciente

Todos

Para Você

98%



Dr. Ricardo Mendes

Clínica Sorriso

Localização 95%

Especialidade 100%

Criar Match

Detalhes do Dentista

92%



Dra. Juliana Costa

Centro Odonto

Localização 88%

Especialidade 96%

Criar Match

Detalhes do Dentista

85%



Dr. Felipe Oliveira

Oral Care

Localização 82%

Especialidade 88%

Criar Match

Detalhes do Dentista

Filtros

Localização

Selecione...

Especialidade

Selecione...

Disponibilidade

Selecione...

Buscar



Matches Recentes

Últimas solicitações

Todos

Pendentes

Confirmados

CS

Carlos Silva

Dr. Ricardo Mendes

15/01/2024

Confirmado

LF

Lucia Ferreira

Dra. Juliana Costa

14/01/2024

Pendente

PS

Pedro Santos

Dr. Felipe Oliveira

13/01/2024

Cancelado

ML

Marina Lima

Dr. Roberto Alves

12/01/2024

Confirmado

Ver Histórico Completo



Figura 3 — Módulo de match com foco no pareamento entre pacientes e dentistas voluntários.

Inbox 32

WHATSAPP

- WhatsApp - Doações 12
- WhatsApp - Jurídico 5

EMAIL

- Email - Atendimento 8
- Email - Financeiro 3

SMS

- SMS - Alertas Urgentes 4
- SMS - Comunicados Gerais 0

Buscar notificações...

Filtrar

HOJE



Ana Clara Souza 09:42

Urgente: Paciente precisa de ajuda

Olá, preciso de ajuda com um paciente que precisa...

Urgente Pacientes



Instituto Jurídico Dental 08:15

Processo nº 12345/2024

Referente ao processo de cobrança...

Jurídico

ONTEM



Campanha Sorriso Social 17:30

Campanha dia 15/12

Venha participar da nossa campanha...

Doações



Fornecedor OrthoSupply 14:20

Entrega confirmada

Sua encomenda foi enviada...

Logística



Selecione uma notificação

Clique em uma mensagem para ver os detalhes

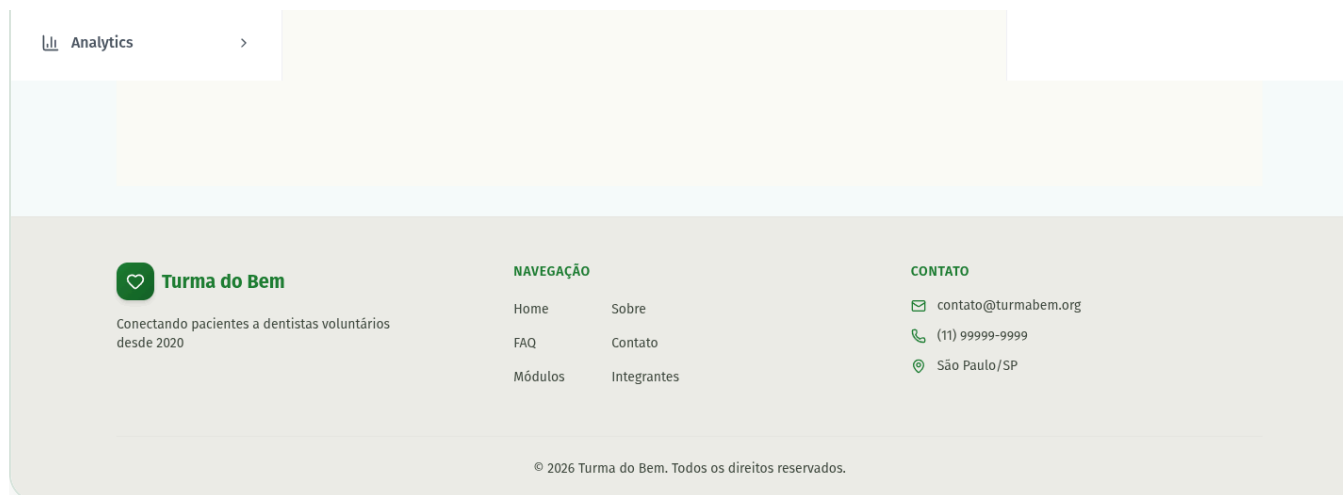


Figura 4 — Centro de comunicação com mensagens e notificações utilizadas na operação da ONG.



Gerenciamento de Usuários

Painel administrativo para visualização e gestão de usuários da plataforma.

Total8

Ativos5

Inativos2

Pendentes1

Pacientes4

Voluntários3

Admins1

Buscar

Buscar por nome ou email...

Tipo

Todos

Status

Todos

Limpar Filtros

USUÁRIO	TIPO	STATUS	CADASTRO	ÚLTIMO ACESSO	AÇÕES
AS Ana Silva ana.silva@email.com	Paciente	Ativo	14/01/2024	19/03/2024	
CS Carlos Souza carlos.souza@email.com	Voluntário	Ativo	31/01/2024	18/03/2024	
MS Maria Santos maria.santos@email.com	Paciente	Pendente	04/03/2024	04/03/2024	
JO João Oliveira joao.oliveira@email.com	Voluntário	Inativo	09/12/2023	14/01/2024	
FL Fernanda Lima fernanda.lima@email.com	Administrador	Ativo	19/11/2023	20/03/2024	
PC Pedro Costa pedro.costa@email.com	Paciente	Ativo	19/02/2024	17/03/2024	
JM Juliana Martins juliana.martins@email.com	Voluntário	Ativo	09/01/2024	20/03/2024	
RM Ricardo Mendes ricardo.mendes@email.com	Paciente	Inativo	14/10/2023	19/12/2023	



Figura 5 — Gestão de usuários com listagem e administração de perfis da plataforma.



Registrar Atendimento

Registre os pacientes atendidos e seus tratamentos realizados

+ Novo Atendimento

DADOS DO PACIENTE

Nome Completo *

Digite o nome do paciente

Idade *

Ex: 25

Telefone *

(11) 98765-4321

Email

paciente@email.com

Endereço

Rua, número, bairro, cidade

DADOS DO ATENDIMENTO

Data do Atendimento *

dd / mm / aaaa

Próxima Consulta

dd / mm / aaaa

Procedimento Realizado *

Selecione...

Condição / Diagnóstico *

Selecione...

Gravidade da Condição *

Leve

Moderada

Grave

Observações e Detalhes do Atendimento

Descreva os detalhes do atendimento, procedimentos realizados, orientações dadas ao paciente...

Status do Tratamento *

Concluído

Em Tratamento

Aguardando

Registrar Atendimento

Atendimentos Registrados

2 registros



Ana Silva

28 anos

Concluído



Restauração



Cárie

moderada



2024-01-15



Paciente relatou dor ao mastigar. Realizada restauração no molar superior direito.



Próxima: 2024-02-15



Carlos Souza

35 anos

Concluído



Limpeza



Gengivite

leve



2024-01-10



Limpeza profissional realizada. Orientado sobre técnica de escovação.



Próxima: 2024-04-10

Resumo

Total de Atendimentos:

2

Concluídos:

2

Em Tratamento:

0

Aguardando:

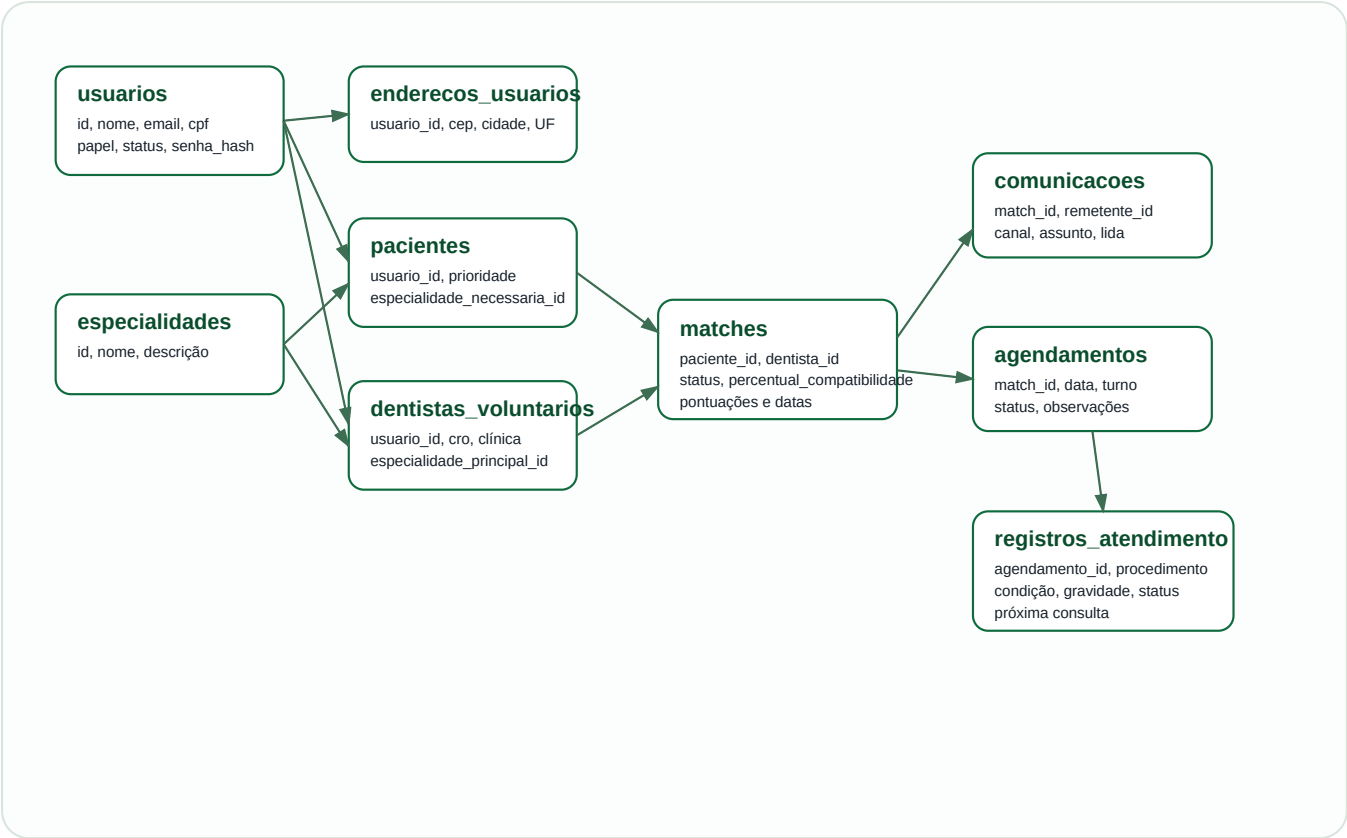
0



Figura 6 — Tela de relatórios e registro de atendimento clínico.

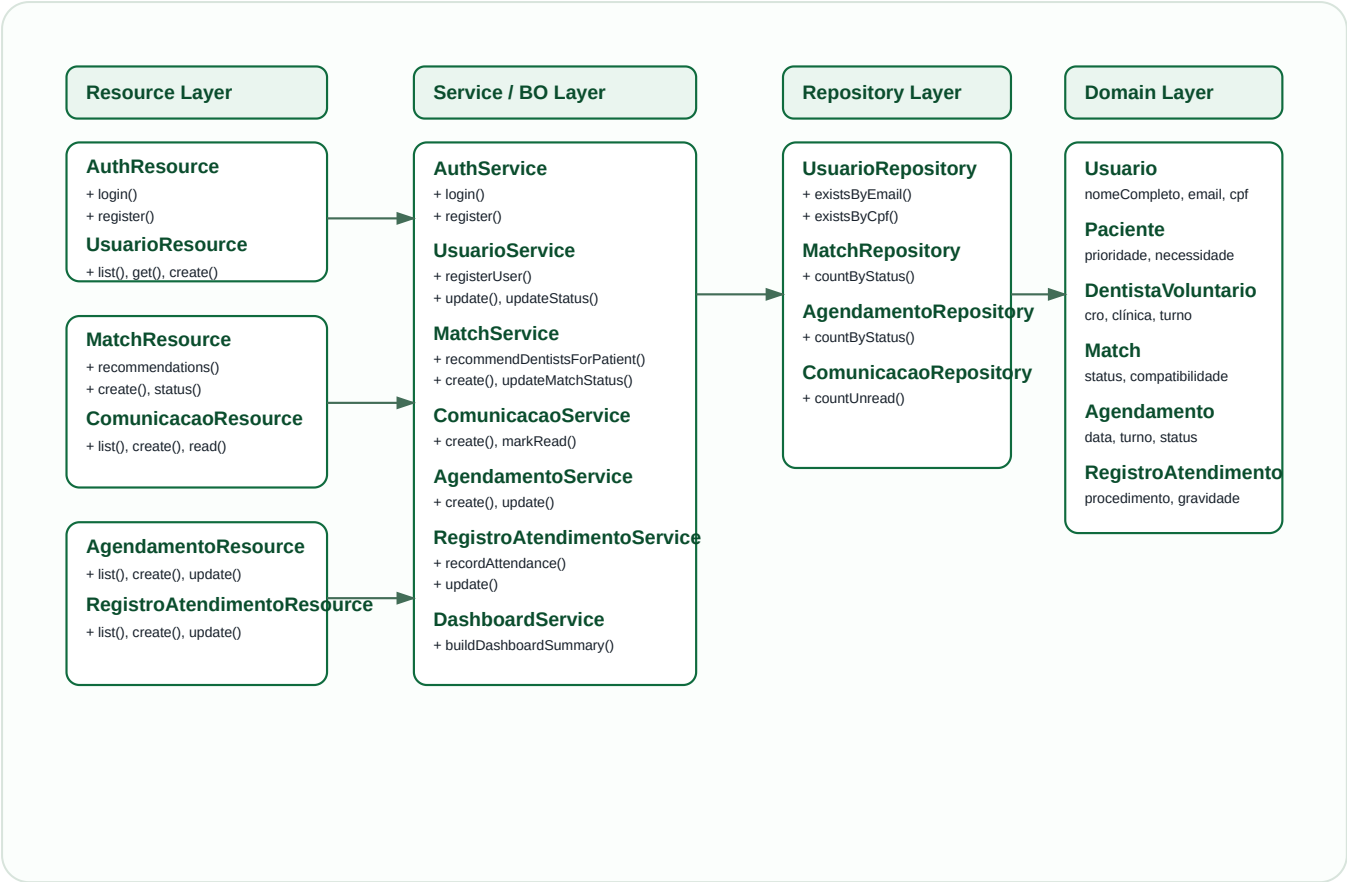
6. Modelo Entidade-Relacionamento (MER)

O banco de dados PostgreSQL foi modelado para representar o fluxo operacional completo da ONG: usuários, perfis específicos, matches, comunicação, agenda e histórico clínico. O diagrama abaixo resume as entidades centrais e seus relacionamentos principais.



7. Diagrama de classes atualizado

A modelagem orientada a objetos foi organizada em camadas. As entidades representam o domínio, os repositórios centralizam o acesso a dados, os serviços concentram as regras de negócio e os recursos REST expõem a API. O diagrama abaixo resume a estrutura principal da solução.



8. Procedimentos para rodar a aplicação

Este repositório corresponde à entrega do backend em Quarkus.

Para executar localmente, é necessário ter uma instância PostgreSQL já criada e acessível.

A aplicação não gera o schema automaticamente

(`quarkus.hibernate-orm.database.generation=none`) e este repositório não inclui frontend, arquivos Docker Compose ou scripts SQL de criação.

8.1 Pré-requisitos

- Java 17
- PostgreSQL em execução
- Schema do banco já existente
- Variáveis de ambiente configuradas, se necessário

Configuração de referência documentada no projeto:

Banco: `turma_do_bem`

Usuário: `turma_admin`

Senha: `turma_segura_2026`

Porta da API: 8080

8.2 Backend

Executar a partir da raiz deste repositório:

```
./mvnw test
```

```
./mvnw verify
```

```
./mvnw quarkus:dev
```

Configurações relevantes do Quarkus:

```
quarkus.datasource.jdbc.url=jdbc:postgresql://localhost:5432/turma_do_bem
```

```
quarkus.datasource.username=turma_admin
```

```
quarkus.datasource.password=turma_segura_2026
```

```
quarkus.http.port=8080
```

```
quarkus.hibernate-orm.database.generation=none
```

```
quarkus.smallrye-openapi.path=/q/openapi
```

```
quarkus.http.cors=true
```

GitHub do projeto: <https://github.com/Carlos-Bianchi/java-challenge-04>

9. Conclusão

A entrega final do projeto Java consolida uma API RESTful completa, conectada ao modelo relacional do sistema e compatível com a arquitetura SPA do frontend. O backend atende aos requisitos acadêmicos ao apresentar múltiplas entidades de domínio, CRUD completo para os recursos essenciais, validações, regras de negócio, tratamento de exceções e integração com PostgreSQL.

Além do aspecto técnico, a solução está alinhada ao problema de negócio da ONG Turma do Bem: organizar o processo de conexão entre pacientes e dentistas voluntários, ampliar a rastreabilidade da operação e oferecer uma base segura para evolução futura com integração real entre as camadas da aplicação.